Receipt Aware Dual State Database Auditor

Reconciliation before checkpointing

Swarnava Chaudhuri 25BCE2463
Shivani Patodia 25BCE2403
Shreya Verma 25BCE2425

Faculty guide Dr. Saritha Murali

A matching audit log can still be wrong

Original record	A fee entry contains 250.00.
Database attack	An administrator changes the row to 999999.00 and rewrites its audit event.
Verification problem	The row and local history agree. A separately held receipt still records 250.00.

The demonstrated trust boundary

Attacker controls	Business rows and the database audit table.
Verifier relies on	An unchanged witness store containing accepted transitions and checkpoints.
Local prototype	Two SQLite files simulate the boundary. Full control of the computer defeats that assumption.

The receipt records an accepted transition

Before the write	Check the latest witnessed revision against the live row.
Witness first	Retain record ID, old state, new state, revision and previous receipt hash.
Business commit	Write the row and matching audit event. A partial commit remains visible as inconsistent evidence.
Later checkpoint	Reconcile evidence, then build a Merkle root over pending receipts.

Verification returns a specific state

Condition	Verdict
Witness and row agree, checkpoint absent	PENDING_ANCHOR
Receipt and Merkle checkpoint agree	VERIFIED
Live row differs from witnessed state	TAMPER_DETECTED
Database audit differs from receipt	AUDIT_TAMPERED
Witnessed deletion has a valid proof	DELETED_VERIFIED

The candidate contribution has a narrow scope

Established work	Merkle proofs, external digests, receipts and asynchronous database integrity already have prior art.
Candidate workflow	Reconcile revision-linked receipts, local audit evidence and live state before accepting a write or sealing a batch.
Patent status	This is a direction for claim-by-claim assessment. The prototype does not establish novelty or non-obviousness.

Sources: US20210377037A1 and US10261711B1. Full references in the report.

A four minute live demonstration

1 Save and inspect	Create two rows. Show PENDING_ANCHOR, then anchor and inspect a real Merkle path.
2 Update normally	Change the amount through Save with receipt. It returns to pending, then verifies after anchoring.
3 Rewrite evidence	Inject a row-and-audit rewrite. Show AUDIT_TAMPERED.
4 Try to anchor	Checkpoint creation returns ANCHOR_BLOCKED. The system refuses to seal the disagreement.

32 tests passed in the recorded run

16 original tests	Canonical hashes, amount handling and ordered Merkle proofs.
16 added tests	Pending changes, coordinated rewrites, deletion, missing evidence, rollback, restart and proof damage.
Reproducible evidence	The source includes the test log, experiment script and machine-readable measurements.

The measurements expose a scaling limit

Records	Median write	Checkpoint	Median verify
10	5.519 ms	8.474 ms	0.509 ms
100	6.085 ms	110.088 ms	1.061 ms
300	6.842 ms	4279.445 ms	12.059 ms

Receipt and checkpoint scans dominate the larger run. Only the Merkle path calculation has logarithmic depth.

One run per size. Python 3.13.15, Windows, synthetic SQLite data. No network or PostgreSQL baseline.

What this prototype establishes

Working result	The tested faults produce distinct verdicts, and mismatched evidence blocks checkpoint creation.
Engineering limits	Local witness simulation, manual checkpointing and full-history scans. The separate PostgreSQL pipeline remains unvalidated here.
Next work	Remote authenticated witness, failure recovery, indexed verification and a formal patent search through the institute.

References and project evidence

Allen et al. Veritas, CIDR 2019
Srivastava et al. Verity, arXiv:1901.00228
US20210377037A1 Asynchronous relational data integrity
US10261711B1 Proof of non-tampering for stored data
Microsoft Learn SQL Server ledger overview
Project source, test log and experiments.json

See the project report for complete bibliographic details and links.